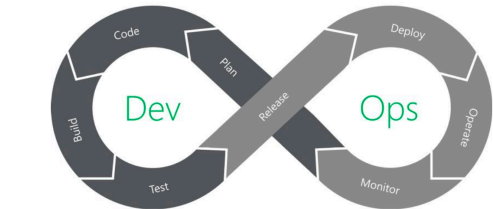


1. CLOUD COMPUTING & DEVOPS

- On-demand Self-Service:** Autom. Bereitstellung von Rechenleistung/Speicher ohne menschliche Interaktion mit dem Provider.
- Broad Network Access:** Zugriff über das Netzwerk mittels Standardmechanismen für div. Clients (Mobil, Laptop).
- Resource Pooling:** Multi-Tenancy; dynamische Zuweisung von Ressourcen je nach Bedarf (Standortunabhängig).
- Rapid Elasticity:** Schnelle, oft automatische Skalierung (Scale-in/out) je nach aktuellem Ressourcenbedarf.
- Measured Service:** Pay-as-you-go; autom. Steuerung, Optimierung & Transparenz durch Messung der Nutzung.

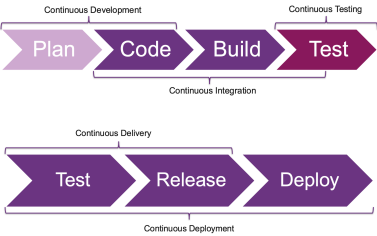
- Web-Services & Container**
- Web-Fehler:** Bugs, Deployment, Skalierung, Ausfälle, Monitoring, Security.
- Container-Issues (2025):** Kultur (47%), Training/Security (36%), CI/CD/Monitoring (35%), Komplexität (34%).

Phase	Aktivitäten	Tools
Plan	Requirements, Roadmaps, Architektur, Security-Pläne	Jira, Trello
Code	Code schreiben, lokale Tests, Peer Reviews, Gated Merges	GitLab, GitHub
Build	Kompilieren, Container-Images, Artefakte pushen	GitLab CI, Docker
Test	Unit-/Integrationstests, E2E, Performance, Security	JUnit, SonarQube
Release	Verifizierte Artefakte pushen, Tagging, Release Notes	Artifactory, Cosign
Deploy	Manifeste anwenden, Strategien, Post-Validierung	Argo CD, Terraform
Operate	Infra betreiben, Skalierung, Patches, Backups	Vault, Velero
Monitor	SLIs/SLOs, Dashboards, Alerts, Logs korrelieren	Prometheus, Datadog



2. CI/CD & DEPLOYMENT

- CI vs. CD**
- CI (Cont. Integration):** Autom. Build/Test bei Push (Plan, Code, Build, Test).
- CDel (Cont. Delivery):** Autom. bis Staging, Prod-Deploy manuell.
- CDep (Cont. Deployment):** Vollautom. Deploy in Prod ohne Mensch.



- Deployment-Strategien**
- Recreate / Big-Bang:** Alle alten Instanzen stoppen, dann neue starten (Downtime).
- Rolling:** Schrittweiser Austausch aller Instanzen (no downtime).
- Blue-Green:** 2 identische Umgebungen (aktiv/inaktiv), schneller Switch.
- Canary:** Zuerst für kleine, spezifische Usergruppe ausrollen.
- Feature Flags:** Inaktiv deployt, per Software-Schalter aktiviert.
- Dark Launching / Shadow:** Hintergrund-Test mit Live-Traffic / Kopie (User sieht nichts).



3. PLATFORM ENGINEERING

- Cultural Mindset:** Design, Bau & Betrieb interner Developer-Plattformen -> viel Cognitiver Load weil man alles selber machen muss. In grossen Firmen wird das durch Plattformengineering als Service bereitgestellt.
- Core Responsibilities:**
 - Self-Service ermöglichen
 - Runtimes & Tooling standardisieren

- Strukturiert, reproduzierbar
- Compliance & Security erzwingen
- Observability bieten

4. INFRASTRUCTURE PROVISIONING (IAC)

- 1. Konzepte**
- IT-Infra:** Compute (VMs, Container), Storage (Block, Object), Network (VPCs, Firewalls, LBs).
- Provisioning (IaC):** Fundament aufbauen (z.B. leere VM). Tools: OpenTofu, Terraform.
- Config. Management (CM):** System einrichten (Pakete, User). Tool: Ansible.
- 2. IaC Vorteile**
- Automatisierung:** Schneller, weniger Fehler.
- Konsistenz:** Identische Umgebungen (Dev, Prod).
- Idempotenz:** Mehrfaches Ausführen führt zum selben Zielzustand ohne Seiteneffekte.
- Versionierung:** Code im Git (PRs, CI/CD).
- Dokumentation:** Code ist die „Source of Truth“.

Merkmal	Imperativ	Deklarativ
Fokus	Wie? (Schritt-für-Schritt)	Was? (Zielzustand)
Zustand	Skripte failen oft (nicht idempotent)	Nur nötiges ändert sich (idempotent)
Tools	Bash, AWS CLI	OpenTofu, Terraform

- 4. OpenTofu / Terraform Toolchain**
- Language:** HCL (HashiCorp Config. Language).
- State** (.tfstate): Mappt den HCL-Code (Desired State) zur Realität. **Bsp:** Verknüpft den internen TF-Ressourcenamen mit der echten Cloud-ID (z.B. ARN). TF weiss so, welche Instanz zuvor erstellt wurde, und prüft anhand der ID die Cloud auf Änderungen.

- Liest NEW (code)/EXISTING(cloud resources)/PREVIOUS(tf state).
- Backend:** Speicherort des States. Remote (S3, GitLab) verhindert Konflikte via **State Locking**.
- Provider:** Plugins für Cloud-APIs (AWS, Azure).

5. Terraform Workflow



```
6. HCL Syntax
// provider: Definiert Ziel-Cloud (z.B. AWS) & Konfiguration (Region).
provider "aws" { region = "eu-central-1" }

// resource: Erstellt eine neue physische/virtuelle Komponente in der Cloud.
resource "aws_instance" "web" { ami = "ami-12345"; instance_type = "t2.micro"

// data: Liest bestehende Ressourcen (read-only) aus der Cloud ein.
data "aws_ami" "ubuntu" { most_recent = true }

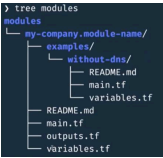
// variable: Inputs (Parameter) zur Wiederverwendbarkeit. Zugriff via var.count
variable "count" { type = number; default = 2 }

// locals: Lokale Worte/Ausdrücke zur Vermeidung von Redundanz. Zugriff via local.name
locals { env_name = "prod-${var.count}" }

// output: Gibt nach dem Apply Infos auf der Konsole aus (z.B. IP).
output "ip" { value = aws_instance.web.public_ip }

// backend: Definiert, wo der tfstate gespeichert wird (z.B. S3).
backend "s3" { bucket = "my_buckets" }

// module: Fasst Ressourcen zusammen (wie Funktionen). Fördert Wiederverwendbarkeit.
module "vpc" { source = "../modules/vpc" }
```

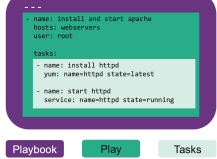


5. CONFIGURATION MANAGEMENT (ANSIBLE)

- 1. Konzepte & Modelle**
- CM:** Einrichtung/Wartung von Software & OS auf bestehender Infra (z.B. Ansible „richtet das Haus ein“, Terraform „baut das Haus“).
- Push-Modell (Ansible):** Control Node drückt Config via SSH auf Zielsysteme (agentless).
- Pull-Modell (Puppet, Chef):** Agent auf Zielsystem holt Config von zentralem Server.
- Idempotenz:** Mehrfache Ausführung = exakt gleiches Ergebnis.
- Config. Drift:** Manuelle Abweichungen vom Soll-Zustand; CM-Tools erzwingen Desired-State.

- 2. Ansible Core**
- Control Node:** Rechner mit installiertem Ansible.
- Managed Nodes:** Zielserver (brauchen nur Python).
- Inventory:** Datei mit IPs/Hostnamen, gruppiert.
- [webservers]** 192.168.1.10
- Modules:** Python-Skripte für Tasks (z.B. apt, copy, service).

3. Task, Play & Playbook



4. Playbook Syntax

```
- name: Setup Webserver
hosts: webservers
become: true # sudo
vars: { nginx_port: 80 }
tasks:
  - name: Install Packages (Loop Example)
    ansible.builtin.apt: { name: "{{ item }}", state: present }
    loop: [nginx, curl]
  - name: Copy Config
    ansible.builtin.template: { src: nginx.conf.j2, dest: /etc/nginx.conf }
  - name: Restart Nginx # Triggert Handler
    notify: [config] # Tag-Beispiel
  handlers:
    - name: Restart Nginx
      ansible.builtin.service: { name: nginx, state: restarted }
```

- Handlers:** Spezielle Tasks (nur 1x am Ende), laufen nur wenn durch notify getriggert.
- Loops (loop):** Führt einen Task iterativ über eine Liste aus (siehe oben).
- Roles:** Strukturieren Playbooks in wiederverwendbare Ordner (tasks, vars, templates).
- Tags:** Markieren Tasks, um gezielt nur diese auszuführen (CLI: --tags config).
- Jinja2-Templating:** Dynamische Dateien & Variablen-Einsatz (z.B. {{ port }}).
- Collections:** Größere Pakete, die mehrere Roles, Modules und Plugins bündeln (Download via ansible-galaxy).

```
vars.yml (Daten):
hostname: Router1
interfaces:
  - name: 1
    ip: 10.3.255.106
    mask: 255.255.255.0
    loopback: false
  - name: 0
    ip: 0.6.6.6
    mask: 255.255.255.255
    loopback: true
  - name: 1
    ip: 192.168.6.1
    mask: 255.255.255.0
    loopback: true

Jinja2-Template (.j2):
<hostname>{{ hostname }}</hostname>
<interface>
  {% for interface in interfaces %}
  <loopback>
    {% else %}
    <loopbackifethernet>
    {% endif %}
    <name>{{ interface.name }}</name>
    <ip>
      <address>
        <primary>
          <address>{{ interface.ip }}</address>
          <mask>{{ interface.mask }}</mask>
        </primary>
      </address>
    </ip>
    {% if interface.loopback %}
    {% else %}
    <loopbackifethernet>
    {% endif %}
    {% endif %}
  </interface>
```

5. Ansible CLI Commands

- Ping:** ansible all -i inventory.ini -m ping
- Ad-Hoc:** ansible webservers -i inventory.ini -m apt -a "name=curl state=present" --become
- Playbook:** ansible-playbook -i inventory.ini site.yml
- Syntax:** ansible-playbook -i inventory.ini site.yml --syntax-check
- Dry-Run:** ansible-playbook -i inventory.ini site.yml --check

6. GITLAB CI/CD

- 1. Core Konzepte**
- Stages:** Phasen der Pipeline (z.B. build, test, deploy). Laufen nacheinander ab.
- Jobs:** Die tatsächlichen Aufgaben in einer Stage. Laufen parallel.
- Runners:** Die Server/Agents (oft Container), die den Code der Jobs ausführen.
- Tags:** Weisen einem Job gezielt spezifische Runner zu (z.B. Runner mit GPU oder MacOS).
- Steps (script):** Einzelne Befehle (z.B. Bash-Commands), die im Job ausgeführt werden.
- Caches:** Speichern Abhängigkeiten (z.B. node_modules) um nachfolgende Pipeline-Runs zu beschleunigen. Bei Pipelinefail bleibt er bestehen -> sonst wird er überschrieben
- Artifacts:** Ergebnisse eines Jobs (z.B. Binaries, Reports), die an spätere Stages weitergegeben werden.
- Dependencies:** Steuern, welche Artifacts aus vorherigen Jobs heruntergeladen werden (Standard: alle).
- Rules:** Bedingungen (z.B. if-Statements), die steuern, ob ein Job überhaupt ausgeführt wird.
- Registries:** Speicherort für fertige Docker-Images (Container Registry) oder Pakete (npm, maven).
- Services:** Zusätzliche Container (z.B. eine Datenbank), die während des Jobs nebenbei laufen (z.B. für Tests).
- Environments:** Definition einer Zielumgebung (z.B. production), dient dem Deployment-Tracking.

```
2. Pipeline Beispiel (.gitlab-ci.yml)
stages: [build, test, deploy]

# Globaler Cache (wird über Pipeline-Runs hinweg geteilt)
cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths: [node_modules/]

build_job:
  stage: build
  image: node:18 # Docker-Image für den Runner
  tags: [docker] # Weist Job einem Runner mit Tag 'docker' zu
  script: # Die Steps
    - npm install
    - npm run build
  artifacts: # Gibt den /dist Ordner an den deploy_job weiter
  paths: [dist/]
  expire_in: 1 week

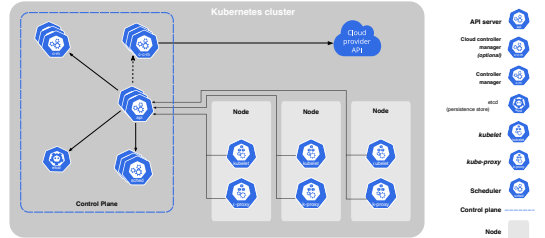
test_job:
  stage: test
  image: node:18
```

```
services: # Startet eine DB parallel für Integrationstests
  - name: postgres:13
    alias: db
script:
  - npm test
rules: # Job läuft nur bei Commits in den 'main' Branch
  - if: '$CI_COMMIT_BRANCH == "main"'

deploy_job:
  stage: deploy
  dependencies: [build_job] # Lädt nur Artifacts vom build_job herunter
  environment: [production] # Tracking in GitLab UI unter Deployments -> Environments
  name: production
  url: https://my-app.com
  script:
    - echo "Deploying $CI_COMMIT_SHA to production"
  rules: # Erfordert einen manuellen Klick in der GitLab UI zum Starten
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: manual
```

7. KUBERNETES (K8S) & ARCHITEKTUR

- 1. Control Plane (Master Node)**
- Steuert den Cluster und speichert den Zustand.
- API Server:** Zentraler Eingangspunkt. Alle Kommunikation läuft hierüber (CLI, Nodes).
- etcd:** Hochverfügbarer Key-Value Store. Speichert den kompletten Cluster-Zustand.
- Scheduler:** Weist neu erstellte Pods den passenden Worker Nodes zu (Ressourcen-basiert).
- Controller Manager:** Überwacht den Ist-Zustand und gleicht ihn an den Soll-Zustand an.
- 2. Worker Nodes**
- Führen die eigentlichen Applikations-Workloads aus.
- Kubelet:** Der Agent auf dem Node. Nimmt Befehle vom API-Server und steuert die Container.
- Kube-Proxy:** Regelt das Netzwerk und das Load-Balancing für die Pods auf dem Node.
- Container Runtime:** Führt die Container tatsächlich aus (z.B. containerd).
- Pod:** Kleinste K8s-Einheit. Kapselt 1-N Container (teilen sich IP & Storage).

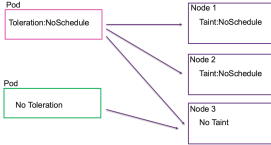


3. QoS Classes (Quality of Service)

- Guaranteed (Prio 1):** Request = Limit für CPU & Mem exakt gleich für alle Container.
- Burstable (Prio 2):** Mind. 1 Container hat Request < Limit (oder Limits fehlen teils).
- BestEffort (Prio 3):** Keine Requests/Limits definiert. Werden bei Knappheit zuerst beendet.

4. Deployment Update Strategien

- Recreate:** Alte Pods löschen, dann neue erstellen. (Downtime! Nötig, falls Versionen nicht parallel laufen dürfen).
- RollingUpdate (Default):** Aktualisiert Pods nacheinander für Zero-Downtime.
 - maxUnavailable: Max. Anzahl an Pods, die während dem Update offline sein dürfen.
 - maxSurge: Max. Anzahl an zusätzlichen Pods, die parallel erstellt werden dürfen (über desired count).
- 5. Taints & Tolerations**
- Taints:** Werden auf Nodes gesetzt und „stoßen“ unerwünschte Pods ab.
- Tolerations:** Werden auf Pods gesetzt und erlauben (erzwingen aber nicht) das Scheduling auf Nodes mit passendem Taint.



6. Probes (Health Checks)

- readinessProbe:** Ist der Pod bereit für Traffic? (Pod ip wird erst bei Erfolg an Services weitergeleitet).
- livenessProbe:** Läuft die App noch oder ist sie eingefroren? (Startet Pod bei Fehler neu).
- startupProbe:** Für langsame Legacy-Apps. Pausiert liveness-Checks während der langsamen Initialisierung.

7. Deployment YAML & InitContainer

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: my-app }
spec:
  replicas: 2
  selector: { matchLabels: { app: my-app } }
  template:
    metadata: { labels: { app: my-app } }
    spec:
      initContainers: # Laufen sequenziell VOR den normalen Containern (müssen erfolgreich beenden)
        - name: init-setup
          image: busybox
          command: ['sh', '-c', 'echo "Setup done"']
      containers:
        - name: web
          image: nginx:alpine
```

8. K8S PACKAGE & CONFIG MANAGEMENT

1. Helm (Package Manager mit Templating)

- Charts:** So nennt Helm seine Pakete. Enthalten alle YAML-Manifeste (Templates) für eine App.
- Values (values.yaml):** Variablen, die beim Deployment dynamisch in die Templates eingesetzt werden.
- Releases:** Jede laufende Instanz eines Charts auf dem Cluster.

Wichtige Chart .yaml Felder:

- apiVersion:** v2 (für Helm 3), v1 (für ältere Versionen / Backwards-Compatibility).
- appVersion:** Version der deployten App (nur Info, hat keinen Einfluss auf die Chart-Version).
- kubeVersion:** Unterstützte K8s-Versionen (z.B. >= 1.12.1 < 1.14.0). Installation bricht ab, wenn inkompatibel.

Template & Values Beispiel:

```
# values.yaml
replicaCount: 3

# deployment.yaml (Template)
spec:
  replicas: {{ .Values.replicaCount }}
```

2. Kustomize (Config Management ohne Templating)

Das Prinzip: Nutzt reale, valide YAML-Dateien. Man definiert eine „Basis-Konfiguration“ (**Base**) und erstellt für verschiedene Umgebungen (Dev, Prod) spezifische **Overlays**.

- Overlays:** Verändern die Base-Konfiguration umgebungsspezifisch.
- Patches:** Gezielte YAML-Snippets, die bestehende Base-Ressourcen überschreiben/ergänzen.
- Transformers:** Passen Ressourcen automatisiert an (z.B. **allen** Ressourcen ein bestimmtes Label anfügen).
- Components:** Wiederverwendbare, einkapselbare Konfigurationsblöcke (flexibler als Standard-Bases).

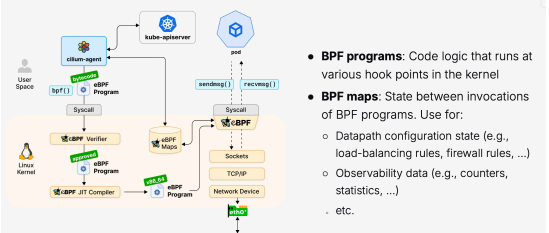
Struktur:	kustomization.yaml:	Patches:
<pre>~/someApp ├── base │ ├── deployment.yaml │ ├── kustomization.yaml │ └── service.yaml ├── overlays │ ├── development │ │ ├── cpu_count.yaml │ │ ├── kustomization.yaml │ │ ├── replica_count.yaml │ │ └── production │ ├── production │ │ ├── cpu_count.yaml │ │ ├── kustomization.yaml │ │ └── replica_count.yaml</pre>	<pre>apiVersion: kustomize.config.k8s.io/v1beta1 kind: Kustomization bases: - ../base namespace: roar-staging # Change the image name & # version images: - name: quay.io/techupskills/ roardb-v2 - name: quay.io/bclaster/ roardb-test newTag: v4</pre>	<pre>patches: - patch: - op: replace path: /metadata/name value: nginx-server target: kind: Service name: nginx-app ns: prod toPorts: - port: "80" protocol: TCP rules: # L7 HTTP-Filterung http: - method: "GET" path: "/public/*" egress: - toIDSet: # OR: Egress zu bestimmten IPs - cidr: "192.168.1.0/24" except: - "192.168.1.50/32" - toEndpointSet: # OR: DNS-Zugriff matchLabels: k8s.io.kubernetes.pod.namespace: kube-system k8s-app: kube-dns toPorts: - port: "53" protocol: UDP rules: dns: - matchPattern: "*mycompany.com"</pre>

Components: Overlays mit Components:

ext-storage:	my-settings:	dev:	test:	prod:
<pre>apiVersion: kustomize.config.k8s.io/v1beta1 kind: Component resources: - storage.yaml # add in persistent storage patchesStrategicMerge: - patch_pv.yaml</pre>	<pre>apiVersion: kustomize.config.k8s.io/v1beta1 kind: Component resources: - my-secret.yaml - configMapGenerator: name: my-secret literals: - key = val</pre>	<pre>resources: - ../base namespace: dev components: - components/ext-storage - components/my-settings</pre>	<pre>resources: - ../base namespace: prod # add in persistent storage components: - components/my-settings</pre>	<pre>resources: - ../base namespace: prod # add in persistent storage components: - components/ext-storage</pre>

9. CILIMUM

Linux kernel + eBPF



Automatically Allowed Connections

- Wenn keine NetworkPolicies definiert sind, ist Traffic standardmäßig erlaubt (Default Deny greift erst, sobald eine Policy einen Pod selektiert).
- Ingress/Egress:** Ohne matchende Policy wird nichts blockiert.
- Worker Nodes:** Ingress von K8s Worker Nodes immer erlaubt (verhindert Blockieren von kubelet Liveness/Readiness Probes).
- Host-Network:** Connectivity von Host-Network Pods ist immer erlaubt (Host-Network = trusted).

Policy Layers

- L3 (Network Endpoints):** to/fromEndpoints, to/fromCIDR (Set), to/FQDN.
- L4:** to/fromPort.
- L7:** DNS, HTTP, gRPC, Kafka, Memcached, Cassandra.

Policy Entities & Identities (toEntities/fromEntities)

- world:** Alle Endpoints außerhalb des Clusters.
- cluster:** Alle gemanagten Pod-Endpoints.
- host:** Der lokale K8s Worker Node.
- remote-node:** Externe K8s Worker Nodes.
- kube-apiserver:** Remote Nodes mit laufendem API-Server.

Default Deny Policy

Hebt das „Automatic Allow“ auf durch leere Regeln für Ingress & Egress (blockiert jeglichen Traffic). App-Teams müssen explizite „Allow“-Ausnahmen deklarieren.

Achtung: Blockiert auch Namespace-interne Traffic und Zugriff auf CoreDNS / K8s API!

Deny All: Allow Within Namespace:

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
metadata:
  name: deny-all
spec:
  endpointSelector: {}
  ingress:
  - {}
  egress:
  - {}
```

Feature-Rich Policy:

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
metadata:
  name: feature-rich-policy
spec:
  endpointSelector:
    matchLabels:
      app: backend # Ziel-Pods (AND)
  ingress:
  - fromEndpoints: # OR: Regel 1
    matchLabels: # Pods mit app=frontend
  AND env=prod
  app: frontend
  env: prod
  matchNamespaces: # UND im Namespace
  ns: prod
  toPorts:
  - port: "80"
  protocol: TCP
  rules: # L7 HTTP-Filterung
  http:
  - method: "GET"
    path: "/public/*"
  egress:
  - toIDSet: # OR: Egress zu bestimmten IPs
    - cidr: "192.168.1.0/24"
    except:
    - "192.168.1.50/32"
  - toEndpointSet: # OR: DNS-Zugriff
    matchLabels:
      k8s.io.kubernetes.pod.namespace: kube-system
      k8s-app: kube-dns
  toPorts:
  - port: "53"
    protocol: UDP
  rules:
  dns:
  - matchPattern: "*mycompany.com"
```

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
metadata:
  name: allow-within-namespace
spec:
  endpointSelector: {}
  ingress:
  - fromEndpoints:
    - {}
  egress:
  - toEndpoints:
    - {}
```

Achtung bei Default-Deny: Folgende Verbindungen werden blockiert:

- DNS-Auflösung:** Keine Namensauflösung mehr (CoreDNS Port 53 blockiert).
- Kubernetes API-Server:** Zugriff von Pods auf API-Server unterbunden.

Egress DNS Allow Rule:

```
egress:
- toEndpoints:
  - matchLabels:
    "k8s.io.kubernetes.pod.namespace": kube-system
    k8s-app: kube-dns
  toPorts:
  - port: "53"
  protocol: ANY
  rules:
  dns:
  - matchPattern: "*"

egress:
- method: "GET"
  path: "/public/*"
```

10. SERVICE MESH

Microservices Pros & Cons

Pros:

- Independent Deployment
- Flexibility & Scalability
- Fault Isolation
- Team Autonomy

Cons:

- Operational Complexity
- Distributed System Challenges
- Data Consistency issues
- Higher Resource Usage
- Increased Testing Overhead

Service Mesh Core Features

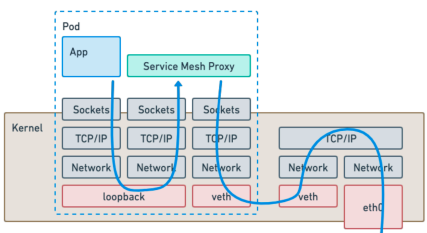
- Traffic Management:** Fine-grained Routing, Traffic Splitting.
 - Resilience:** Fault Tolerance, Automatic Retries & Failback.
 - Security:** Encrypted Communication (mTLS), Authentication, Authorization.
 - Observability:** Metrics, Logs, Tracing.
- Use Cases**
- Canary / Blue-Green:** Stufenweises Traffic-Shifting zwischen App-Versionen.
 - Circuit Breaking:** Verhindert kaskadierende Ausfälle durch schnellen Fail-Fast.
 - Rate Limiting:** Schützt nachgelagerte Services vor Überlastung.
 - Zero-Trust Security:** Verschlüsselt (mTLS) und authentifiziert jeden Aufruf.

Architecture (Data & Control Plane)

- Data Plane:** Leichtgewichtige Proxies (z.B. Envoy) als Sidecars (Kommunikation, Security, Metriken).
 - Control Plane:** Verwaltet Konfigurationen & Policies (z.B. Istio) und verteilt Regeln dynamisch.
- Core Components & Tools:**
- Istio Ingress Gateway:** Envoy Proxy als Einstiegspunkt für den Traffic in das Service Mesh.
 - Istiod:** Backbone der Control Plane (Konfiguration von Proxies und Richtlinien).
 - Jaeger:** Bietet Tracing-Funktionalität für Traffic innerhalb des Service Mesh.
 - Kiali:** Graphisches Dashboard zur Visualisierung von Struktur, Traffic und Health im Mesh.

Istio Sidecar Downsides

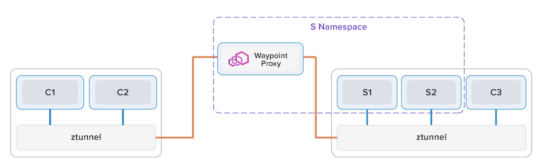
- Invasiveness:** Modifizierte Pod-Spec & umgeleiteter Traffic. Service-Mesh-Upgrades erfordern einen Pod-Neustart.
- Underutilization:** Hohe Overhead-Ressourcen (CPU/Mem) für jeden Workload-Proxy. Ressourcen müssen für Worst-Case-Szenarien provisioniert werden.



Istio Ambient Mode

- Splittert Service-Mesh-Funktionen in zwei Schichten auf (ohne Sidecars):
- ztunnel (Zero-Trust Tunnel):** Läuft als DaemonSet pro Node. Handhabt L4-Transport (mTLS, Verschlüsselung, Routing, Identität, Telemetrie, Metriken) für alle Pods auf dem Node.

- Waypoint Proxy:** Optionaler L7-Proxy (Envoy) pro Namespace / Service-Account. Wird nur bei Bedarf für L7-Policies (HTTP-Routing, Auth, Rate Limiting) zwischen-geschaltet.



K8s Gateway API Routing (HTTPRoute)

HTTPRoute Canary (Traffic Splitting):

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: my-app-route
spec:
  parentRefs:
  - name: my-gateway
  rules:
  - backendRefs:
    - name: frontend
      port: 5000
      weight: 90
    - name: frontend-v2
      port: 5000
      weight: 10
```

HTTPRoute Shadow (Request Mirroring):

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: my-app-mirror
spec:
  parentRefs:
  - name: my-gateway
  rules:
  - backendRefs:
    - name: frontend
      port: 5000
    - type: RequestMirror
      requestMirror:
        backendRef:
          name: frontend-v2
          port: 5000
```

Waypoint Proxy Deployment

- Ein Waypoint-Proxy wird **pro Namespace** (oder Service-Account) deployt. **Wichtig:** L7-Ressourcen (VirtualService, HTTPRoute) greifen im Ambient Mode erst, wenn ein Waypoint-Proxy im Namespace läuft (andernfalls nur L4 per ztunnel).
- CLI:** istioctl waypoint apply -n my-namespace
- YAML (Gateway API):**

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: waypoint
spec:
  spec:
    gatewayClassName: istio-waypoint
  listeners:
  - name: mesh
    port: 15008
    protocol: HBONE
```

11. OBSERVABILITY

Monitoring vs. Logging

Monitoring (Metrics/Trends - Quantitative):

- Utilization (Memory, Disk, CPU, ...)
- Counters (Requests, Error counts, ...)
- Rates (Error Rate, throughput, ...)

Push vs. Pull Monitoring

- Push (Agent-based):** Agent-Software sendet Daten aktiv an Collector.
- Vorteil:** System entscheidet, wann gesendet wird.
- Nachteil:** Agent-Installation nicht überall möglich.

Logging (Detailed Events - Qualitative):

- Stack traces & Exception details
- System messages & Debug logs
- Business events & Audits

Pull (Agentless):

- Collector fragt System ab (HTTP, SNMP, ICMP).
- Vorteil:** Collector steuert Abfragefrequenz.
- Nachteil:** Überwachtes System muss erreichbar sein.

Scaling Prometheus

- Federated Prometheus:** Hierarchische Föderation. Übergeordneter Prometheus (z.B. per Region) scrapte selektierte Metriken von untergeordneten Instanzen.
- Thanos:** Sidecar-basierte Föderation. Bietet globale Abfragen, Langzeitspeicherung (Object Storage) und Deduplizierung.
- Grafana Mimir:** Horizontal skalierbar mit Microservices (Ingestor, Querier). Unterstützt Multi-Tenancy; nutzt Remote-Write (Push).

PromQL Examples

- Counter (total):** Braucht rate(...[5m]), um Geschwindigkeit pro Sekunde zu berechnen.
- Gauge:** Braucht kein rate(), da es bereits den aktuellen Zustand darstellt.

Pods per Namespace:

sum by (namespace) (kube_pod_info) (namespace, pod, container)

count by (namespace) (sum by (namespace, pod, container) (kubernetes.pod_container_info{container!=""})) unless sum by (namespace, pod, container) (kubernetes.pod_container_resource_limits{resource="cpu"})

sum by (namespace) (changes(kubernetes.pod_status_ready{condition="true"}[5m]))

sum by (namespace) (kubernetes.pod_status_ready{condition="false"}[5m]))

sum(kubernetes.pod_status_ready{condition="false"}[5m]))

sum(kubernetes.pod_status_ready{condition="false"}[5m]))

sum(kubernetes.pod_status_ready{condition="false"}[5m]))

sum(kubernetes.pod_status_ready{condition="false"}[5m]))

sum(kubernetes.pod_status_ready{condition="false"}[5m]))

sum(kubernetes.pod_status_ready{condition="false"}[5m]))

sum(kubernetes.pod_status_ready{condition="false"}[5m]))

sum(kubernetes.pod_status_ready{condition="false"}[5m]))

sum(kubernetes.pod_status_ready{condition="false"}[5m]))

sum(kubernetes.pod_status_ready{condition="false"}[5m]))

sum(kubernetes.pod_status_ready{condition="false"}[5m]))

sum(kubernetes.pod_status_ready{condition="false"}[5m]))

Logging Architectures

- Node Level (DaemonSet):** Ein Logging-Agent läuft auf jedem Node.
 - + Funktioniert auch bei Container-Absturz.
 - + Versand ist gebatcht (nicht zwingend Echtzeit).
 - App Level (Direct Push):** Applikation pusht Logs direkt an Backend.
 - + Niedrige Latenz (Echtzeit).
 - + Komplexität in App; Logverlust bei Verbindungsabbruch/frühem Crash.
 - Sidecar:** Separater Logging-Container im selben Pod liest & leitet Logs weiter.
- Grafana Loki & Alloy**
- Loki:** „Prometheus für Logs“. Indexiert nur Metadaten/Labels (nicht den Logtext selbst) -> extrem kleiner Index & sehr kosteneffizient. LogQL für Queries.
 - Alloy:** Universeller Open-Source Telemetrie-Collector (Grafana Agent Nachfolger). Integriert OTel, Prometheus & Loki; sammelt Metrics, Logs, Traces & Profiles deklarativ in einer Pipeline.

LogQL Queries (Loki)

Log Queries (Output: Log-Zeilen):

```
{app="api"} |= "error"
= "176" [1m]
```

Line-Filter (Text-Inhalt):

```
|= : Zeile enthält String.
|= : Zeile enthält String nicht.
|= : Zeile matcht Regex.
|= : Zeile matcht Regex nicht.
```

Metric Queries (Output: Time Series):

```
rate({app="log-generator"} |
= "176" [1m])
```

12. GITOPS & CD

Push-based vs. Pull-based GitOps

Push-based (Imperativ):

- + Einfach zu nutzen & bekannt aus klassischem CI/CD.
- + Schnelle Deployment-Geschwindigkeit.
- Erfordert offene Firewall/Admin-Rechte für externes CI/CD.
- Pipeline-Anpassung bei jeder neuen Umgebung nötig.
- configuration Drift ist schwerer zu kontrollieren.

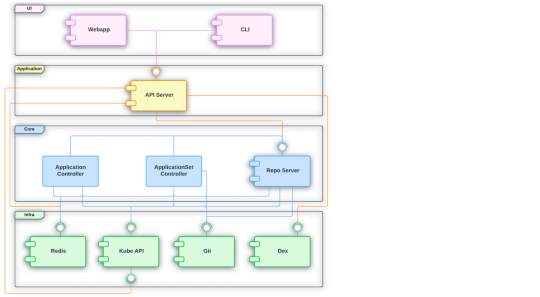
Pull-based (Deklarativ / GitOps):

- + Sicherer: Keine offene Firewall / externe Admin-Rechte nötig.
- + Änderungen werden autom. erkannt/angewendet (Drit-Korrektur).
- + Einfache Skalierung identischer Cluster.
- Benötigt In-Cluster Agent (z.B. Argo CD).
- Moderate Deploy-Geschwindigkeit (Polling-Interval / Sync).

Argo CD vs. Flux CD

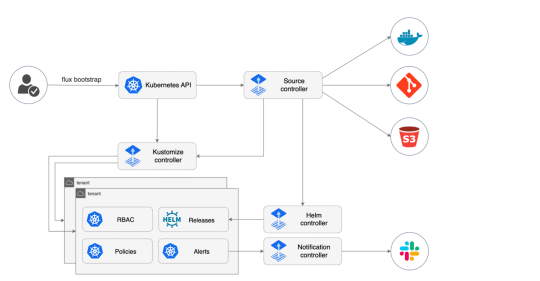
Argo CD:

- Features:** Web UI, Multi-Cluster Management, SSO-Integration, App-of-Apps Pattern.
- Ressourcen:** Application, ApplicationSet, AppProject.



Flux CD:

- Features:** GitOps Toolkit (modulare Controller), native K8s-Integration, CLI-driven.
- Ressourcen:** GitRepository, OCIRepository, Bucket, HelmRepository, Kustomization, HelmRelease.



Secrets Management in GitOps

- Sealed Secrets:** Asymmetrisch. Verschlüsselung erfolgt lokal via kubeseal (Public Key). Entschlüsselung im Cluster durch Controller (Private Key) in native K8s Secrets.
- Mozilla SOPS:** Symmetrische Datei-Verschlüsselung (nur Values verschlüsselt -> Git Diff lesbar). Nutzt KMS (AWS, Vault, etc.) oder PGP/age. Entschlüsselung in CI/CD oder durch In-Cluster Operator.
- External Secrets (ESO):** Pull-Modell. Ein Operator fragt externe Secrets-Manager (Vault, AWS Secrets Manager, etc.) ab und synchronisiert Werte in native K8s Secrets.
- Secrets Store CSI Driver:** Mountet Secrets aus externen Providern direkt als Volume (RAM-Mount via CSI-Speichertreiber) in die Pods. Kein Ablegen im etcd notwendig.